

MutoRS-Specific Large Model Configuration and Related Content Explanation

MutoRS-Specific Large Model Configuration and Related Content Explanation

- Introduction
- Configuring the Startup File
- Start the Dify environment
- Start the Docker environment
- Configure Radar Model
- Speed Calibration
 - 1. Why calibrate the speed?
 - 2. Preparation
 - Hardware Preparation
 - 3. Quick Start
 - 4. Detailed Operation Process
 - Mode 1: Linear Speed Calibration
 - Mode Two: Angular Speed Calibration
 - 5. Data Processing and Activation
 - 6. Verifying Results

Introduction

In previous chapters, we learned some general configuration basics for large AI models. In MutoRS, these configuration methods are slightly different. This tutorial will explain in detail how to configure the MutoRS large model.

Configuring the Startup File

Here we need to prepare the API Key for the Bailian Large Model Platform, in the format "sk-xxxxxxxxxxxxx".

In VNC, enter the Raspberry Pi mainboard.

Enter the following command:

```
sudo gedit /usr/bin/start_muto_docker
```

```

1 #!/usr/bin/env bash
2
3 # =====
4 # User Configuration Area
5 # =====
6
7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 "
12 export ALIBABA_API_KEY="sk- 20"
13
14 export FORCE_VOICE_PROVIDER="xunfei"
15
16 # 1.2 General Voice Settings
17 "
18 export VOICE_LANGUAGE="en"
19
20 # 2. Dify Configuration
21 # Auto-detect IP from wlan0
22 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
23 # Fallback if wlan0 has no IP
24 if [ -z "$DIFY_HOST" ]; then
25     export DIFY_HOST="127.0.0.1"
26 fi

```

①: Enter your API Key here

②: Language selection: set to zh_cn for Chinese, en for English

Overseas users need to change this configuration item to "xunfei"

```

7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 # -----
12 export ALIBABA_API_KEY="sk-
13
14 export FORCE_VOICE_PROVIDER="xunfei"
15
16 # 1.2 General Voice Settings
17 export VOICE_LANGUAGE="en"
18
19 # 2. Dify Configuration
20 # Auto-detect IP from wlan0
21 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
22 # Fallback if wlan0 has no IP
23 if [ -z "$DIFY_HOST" ]; then
24     export DIFY_HOST="127.0.0.1"
25 fi
26
27 if [ "$VOICE_LANGUAGE" = "zh_cn" ]; then

```

After setting, click the save button in the upper right corner to save the script.

Start the Dify environment

On the host machine, enter

```
bringup_dify
```

```
pi@raspberrypi:~ $ bringup_dify
[INFO] 正在获取wlan0网卡的IP地址...
[INFO] 获取到的IP地址: 192.168.12.36
[INFO] 已备份原配置文件到: /home/pi/dify/docker/.env.backup.20260107_113344
[INFO] 发现现有BASE_URL配置, 正在更新...
[INFO] 配置文件更新完成: BASE_URL=http://192.168.12.36:8080
[INFO] 当前.env文件中的BASE_URL配置:
SUPABASE_URL=your-server-url
BASE_URL=http://192.168.12.36:8080
[INFO] 正在启动dify docker服务...
[INFO] 执行: docker compose up -d
WARN[0000] No services to build
[+] up 2/9
✓ Network docker_ssrf_proxy_network Created 0.0s
✓ Network docker_default Created 0.1s
:: Container docker-weaviate-1 Creating 0.3s
:: Container docker-web-1 Creating 0.3s
:: Container docker-sandbox-1 Creating 0.3s
:: Container docker-db_postgres-1 Creating 0.3s
:: Container docker-redis-1 Creating 0.3s
:: Container docker-init_permissions-1 Creating 0.3s
:: Container docker-ssrf_proxy-1 Creating 0.3s
```

If you don't need to start Dify, you can enter the following command to shut it down:

```
shut_dify
```

Start the Docker environment

On the host machine, enter

```
start_muto_docker
```

```
root@raspberrypi:~
Launch Info
Image: 192.168.2.51:5000/muto-humble:latest
Network Mode: host
Service Ports: 80, 9090, 8888, 8080
X11 Env: DISPLAY=:0

Container started successfully! Container ID: 5d7331045e68

>>> Configure camera devices...
>>> Configure audio...
✓ Found Audio Device: (Card 2)
>>> Ready for use...

Access Methods
Enter container: docker exec -it 5d7331045e683414f47c0fa393325c53e27a8a6bf60a9d9b933a6e20f9c5b05b /bin/bash

Container ready!

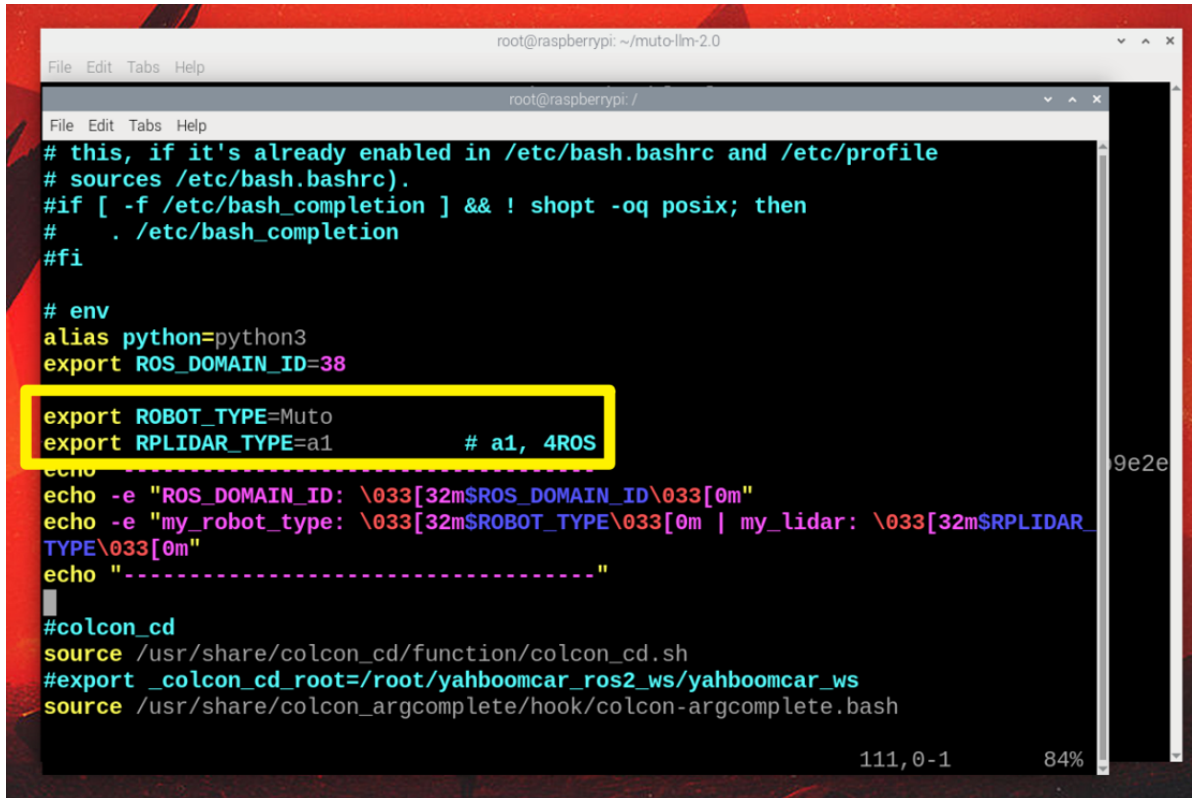
-----
ROS_DOMAIN_ID: 38
my_robot_type: Muto | my_lidar: a1
-----
root@raspberrypi:~#
```

Configure Radar Model

(Configuration is required for 4ROS radar; A1 radar does not require configuration)

Edit the ~/.bashrc file. You can use any editor you prefer, such as vim or nano.

Locate the section shown in the image and select either a1 or 4ROS according to the radar type (note capitalization).



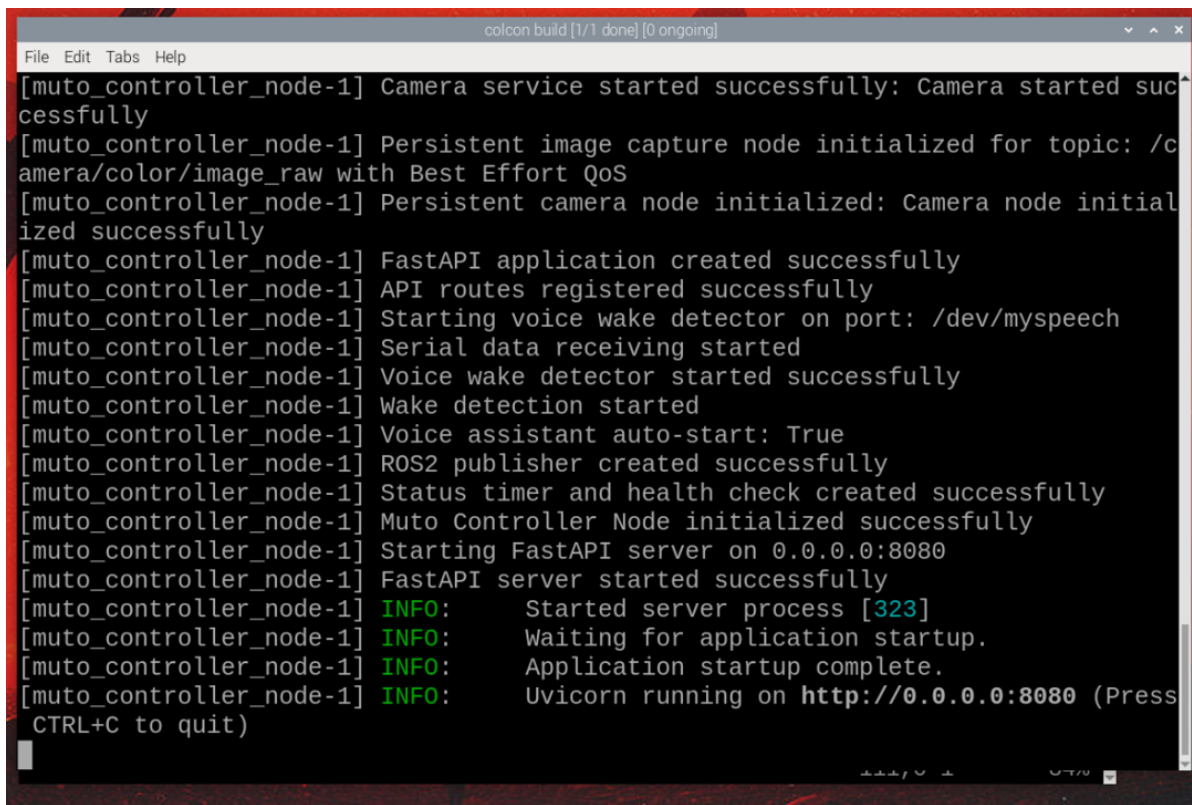
After configuration, you need to source the file.

```
source ~/.bashrc
```

Navigate to the mut0-llm-2.0 directory and run the rebuild_and_launch.sh script.

```
cd mut0-llm-2.0/ && ./rebuild_and_launch.sh
```

The following interface indicates successful startup.



```
colcon build [1/1 done] [0 ongoing]
File Edit Tabs Help
[muto_controller_node-1] Camera service started successfully: Camera started suc
cessfully
[muto_controller_node-1] Persistent image capture node initialized for topic: /c
amera/color/image_raw with Best Effort QoS
[muto_controller_node-1] Persistent camera node initialized: Camera node initial
ized successfully
[muto_controller_node-1] FastAPI application created successfully
[muto_controller_node-1] API routes registered successfully
[muto_controller_node-1] Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] Serial data receiving started
[muto_controller_node-1] Voice wake detector started successfully
[muto_controller_node-1] Wake detection started
[muto_controller_node-1] Voice assistant auto-start: True
[muto_controller_node-1] ROS2 publisher created successfully
[muto_controller_node-1] Status timer and health check created successfully
[muto_controller_node-1] Muto Controller Node initialized successfully
[muto_controller_node-1] Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] FastAPI server started successfully
[muto_controller_node-1] INFO:      Started server process [323]
[muto_controller_node-1] INFO:      Waiting for application startup.
[muto_controller_node-1] INFO:      Application startup complete.
[muto_controller_node-1] INFO:      Uvicorn running on http://0.0.0.0:8080 (Press
CTRL+C to quit)
```

Supplement: Regarding the `rebuild_and_launch` startup script, you can use the `--no-voice` parameter to start the plain text version.

Speed Calibration

1. Why calibrate the speed?

Due to differences in motors, fine-tuning of mechanical structures, and variations in ground friction, the actual movement speed of each hexapod robot will deviate slightly. This script (`speed_calibration_30_levels.py`) is used to establish a precise mapping between **gear levels (Level 1-30)** and **actual physical speed (m/s or deg/s)** through field testing.

If you don't need the Muto to move with extremely precise distance/speed, you can skip this step.

2. Preparation

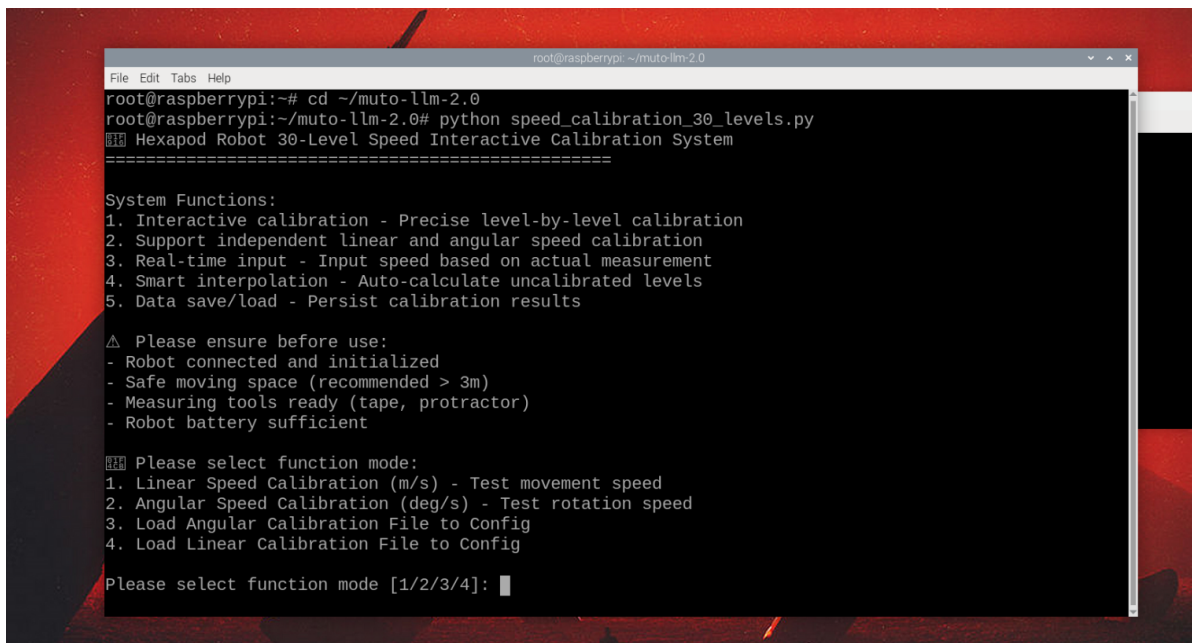
Hardware Preparation

- **Field:** An open, flat surface (3m x 2m or larger recommended). The surface material should be consistent with the actual operating environment (different friction surfaces such as tiles and carpets will greatly affect speed).
- **Tools:** Measuring tape (at least 0.5 meters), marking tape (for marking start and end points).
- **Muto:** Ensure battery is fully charged

3. Quick Start

Ensure you are in the Docker environment. Open a terminal and run:

```
cd ~/muto-11m-2.0
python speed_calibration_30_levels.py
```



```
File Edit Tabs Help
root@raspberrypi: ~/muto-llm-2.0
root@raspberrypi:~# cd ~/muto-llm-2.0
root@raspberrypi:~/muto-llm-2.0# python speed_calibration_30_levels.py
Hexapod Robot 30-Level Speed Interactive Calibration System
=====

System Functions:
1. Interactive calibration - Precise level-by-level calibration
2. Support independent linear and angular speed calibration
3. Real-time input - Input speed based on actual measurement
4. Smart interpolation - Auto-calculate uncalibrated levels
5. Data save/load - Persist calibration results

⚠ Please ensure before use:
- Robot connected and initialized
- Safe moving space (recommended > 3m)
- Measuring tools ready (tape, protractor)
- Robot battery sufficient

📋 Please select function mode:
1. Linear Speed Calibration (m/s) - Test movement speed
2. Angular Speed Calibration (deg/s) - Test rotation speed
3. Load Angular Calibration File to Config
4. Load Linear Calibration File to Config

Please select function mode [1/2/3/4]:
```

The main menu will be displayed after the program starts:

-  Please select a function mode:
1. Linear velocity calibration (m/s) - Test forward, backward, left, and right translation speed
 2. Angular velocity calibration (deg/s) - Test rotation speed
 3. Load angular velocity calibration file into configuration
 4. Load linear velocity calibration file into configuration

4. Detailed Operation Process

Mode 1: Linear Speed Calibration

This mode is used to calibrate the robot's movement speed. The system will select key speed ranges (e.g., 1, 5, 10, 15, 20, 25, 30) for testing.

1. **Enter 1 to enter mode.**
2. **Test Procedure** (for each speed range):
 - **Placement:** Place the robot in the starting position and mark the front of the robot with tape.
 - **Running:** Press **Enter**, and the robot will automatically move forward for **3 seconds** before stopping.
 - **Measurement:** Measure the actual distance traveled (in meters) using a measuring tape.
 - **Input:** Enter the distance value in the terminal (e.g., `0.45`) and press Enter to confirm.
3. **Loop:** The system will automatically switch to the next speed range and repeat the above steps.

Tip: If a test goes astray or a measurement error occurs, enter `u` to undo the previous step; enter `r` to re-enter the distance.

Mode Two: Angular Speed Calibration

This mode is used to calibrate the robot's spin speed.

1. **Enter 2 to enter mode.**
 2. **Test Procedure** (for each gear):
 - **Positioning:** Record the robot's current orientation (as the 0-degree reference line).
 - **Start:** Press **Enter**, and the robot will begin to rotate continuously in place.
 - **Stop:** Observe the robot. When it completes exactly one **full rotation (360 degrees)** and returns to its original orientation, **immediately press the Enter key.**
 - **Record:** The system will automatically calculate the time and angular velocity; no manual input is required.
-

5. Data Processing and Activation

After completing the tests for all key gears, the script will automatically process the data:

1. **Intelligent Interpolation:**
 - When the system asks whether to interpolate, enter **y**.
 - The script uses an algorithm to calculate the speed values for untested gears (middle gears) and completes the data for gears 1-30.
2. **Saving and Applying:**
 - The system will generate a `.json` backup file.
 - **Crucial Step:** When asked whether to write to `robot_config.json`, be sure to enter **y**.
 - Only after writing the configuration will the calibration data take effect on the robot's navigation program.

6. Verifying Results

You can open [config/robot_config.json](#) to view the updated data:

- **Linear Velocity:** View the `linear_speed_calibration_30_levels` field.
- **Angular Velocity:** View the `angular_speed_calibration_30_levels` field.

Example:

```
"linear_speed_calibration_30_levels": {  
  "1": 0.02,  
  "5": 0.08,  
  ...  
  "30": 0.55  
}
```

- If calibrated on a smooth floor but operated on a carpet, the robot may not travel the expected distance due to increased friction, resulting in navigation errors. Please always calibrate in an actual operating environment.

